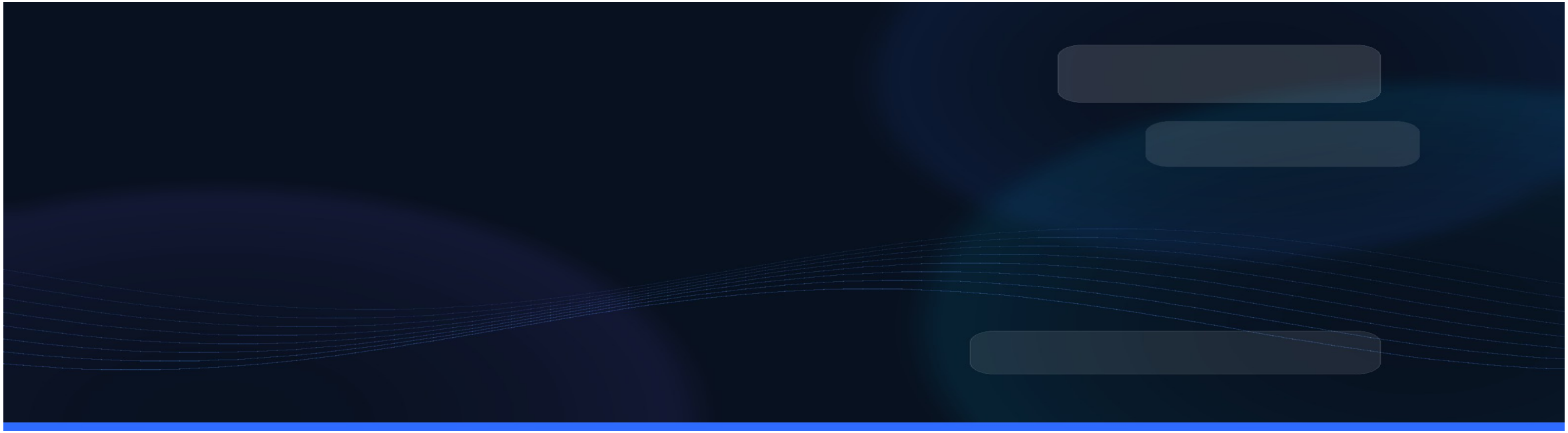


PS5 / PUBLIC EDITION

THE COMPREHENSIVE GUIDE TO TURNING A PS5 INTO THE ULTIMATE HOME CONSOLE

Homebrew foundation • Remote Play to PC • File tools • Emulation • Troubleshooting

13.40	OCT 2026	PUBLIC / PRIVACY-SAFE
FIRMWARE	EDITION	FORMAT

00

START HERE

Read This First

The few things worth knowing before touching the setup.

AI

Use this guide with an AI assistant

You can upload this document to ChatGPT, Claude, or another capable assistant and ask it to walk you through one step at a time. Tell the assistant your firmware and what you currently see on screen. Do not publish or paste private IP addresses, account IDs, pairing PINs, or personal save data unless they are truly necessary for troubleshooting.

SAFE

Ownership and safety

Use only backups, BIOS/firmware files and keys that you legally obtained from hardware and software you own. Back up saves before account activation, save resigning, emulator migration, or storage changes. Avoid unnecessary reboots if your exploit chain is unstable.

This guide records both the clean installation path and the mistakes that caused the most wasted time during a real setup. Items marked “Confirmed” worked in the tested setup. Items marked “Upstream” come from the project’s own documentation. Items marked “Unresolved” are not presented as a guaranteed fix.

CONFIRMED	UPSTREAM	UNRESOLVED
Worked in tested setup	From project docs	Not guaranteed

01

ROADMAP

Priority Map

Build the foundation first; add emulators only after the base is stable.

1

WebKit entry point + WebKit Autoloader

Gets you into the homebrew environment reliably.

2

Payload Manager

Central place to install, launch, import and autoload payloads.

3

etaHEN + kstuff

Provides the homebrew/JIT/sandbox capabilities required by modern PS5 homebrew.

4

Web File Manager + ShadowMountPlus

Lets you manage files remotely and register folder-based homebrew titles.

5

Remote Play to PC

Makes troubleshooting much easier: capture the PS5 screen directly on the PC and share screenshots/logs with an AI.

6

Emulators, one at a time

Install, test, and solve each emulator before moving to the next.

01

FOUNDATION

1. The Base Homebrew Chain

1.1 What each component does

WebKit / exploit entry point

ENTRY

→

Payload Manager

PAYLOADS

→

etaHEN + kstuff

HOMEBREW / JIT

→

ShadowMountPlus + Web File Manager

FILES / TITLES

WebKit / exploit entry point

Initial entry into the jailbreak/homebrew chain. The exact exploit depends on firmware.

Payload Manager

Web dashboard for importing, launching, storing and autoloading payloads.

WebKit Autoloader

Makes the entry point more convenient and can hand off to Payload Manager automatically.

etaHEN

All-in-one homebrew enabler. The Toolbox contains options used by some emulators.

Important: GoldHEN is not the PS5 HEN used here

GoldHEN is a PS4 Homebrew Enabler. On PS5, this guide uses etaHEN + kstuff instead. Do not waste time looking for a “PS5 GoldHEN” package for this setup.

01.2

WEBKIT AUTOLOADER

1.2 Install WebKit Autoloader

Two installation paths, one clean daily entry point.

Already jailbroken

1. Run webkit-autoloader-installer_vX.Y.Z.elf through elfldr or Payload Manager.
2. The installer opens the browser once, caches the local autoloader page, creates the WebKit Autoloader home-screen app, and exits.
3. Perform the one reboot required by the installer, then launch WebKit Autoloader from the home screen.

Not jailbroken yet

1. Run the matching webkit-autoloader-host .exe or .py on a PC on the same network.
2. Temporarily point the PS5 network DNS to that PC’s local address.
3. Open Settings → User’s Guide on the PS5 to run the local installer, then launch the new WebKit Autoloader tile.

Firmware 13.40 note

Relapse is the relevant exploit path on 13.40 and requires an active Wi-Fi or Ethernet interface. Internet access itself is not required.

01.2

AUTOLOADER NOTES

Release files, autoload behavior, and the GitHub trap

Keep this page as the quick reference when downloads start getting confusing.

v0.5.x / 13.40	Payload Manager	ELF / PPSA ZIP
Firmware-aware release	Default hand-off	Installable asset

Autoloader layout

```
USB:/ps5_autoloader/
  pldmgr.elf
  autoload.txt
```

Rule that prevents hours of confusion

On GitHub, the automatically generated “Source code” archives are usually not installable PS5 builds. An installable release normally contains a ready .elf, a release ZIP with a PPSA title folder, or both.

01.3–01.4

CORE SERVICES

Payload Manager + etaHEN

Keep one clean HEN stack and only enable what the app needs.

1.3 Install and use Payload Manager

1. Prefer Payload Manager through the Autoloader. The project recommends using an autoloader, and modern autoloaders include Payload Manager as the default payload.
2. Once Payload Manager is running, open its web dashboard from a device on the same network. http://<PS5_IP>:8084
3. Use Manage Payloads to import payloads from USB. When offered Copy to Internal vs Move to Internal, choose Copy to Internal if you want the USB copy to remain as a backup.
4. Use Autoload only for payloads you intentionally want every session. Avoid stacking multiple versions of the same HEN at once.

1.4 etaHEN for firmware 13.40

1. Download the unified release ELF for firmware 11.00–13.60.
2. Import it into Payload Manager and launch it instead of the older etaHEN build. Do not run two etaHEN versions on top of one another.
3. Open etaHEN Toolbox. Keep Lite Mode off if you need etaHEN services and plugins.
4. Use app jailbreak/JIT options only for titles that explicitly require them.

01.4

ETAHEN 13.40

What was confirmed in the tested setup

The useful lesson is simple: one compatible build, one clean stack, no duplicate config keys.

Confirmed in the tested setup

The old etaHEN 2.5B build exposed incomplete Toolbox behavior on firmware 13.40. Replacing it with the unified 11.00–13.60 build restored the etaHEN Toolbox and exposed the required debug options.

Configuration-file mistake to avoid

If you edit an app-jailbreak list by hand, do not accidentally duplicate the key. Wrong: exact_title_ids=exact_title_ids=... Correct: exact_title_ids=ITEM00001,...

Lite Mode

Keep it off when you need etaHEN services and plugins.

App Jailbreak / JIT

Enable only for titles whose developers explicitly require it.

01.5

FILE & TITLE TOOLS

1.5 kstuff, Web File Manager and ShadowMountPlus

Three tools that keep the console manageable without unplugging storage.

kstuff

kstuff: keep a firmware-compatible build available and load it before software that requires it.

Web File Manager

Web File Manager: launch it from Payload Manager, then open its web UI from another device. In the tested setup it listened on port 8888.

ShadowMountPlus

ShadowMountPlus scans internal and external homebrew locations. Useful public scan roots include /data/homebrew and /mnt/usb0/homebrew through /mnt/usb7/homebrew. For folder-based apps, keep the full title folder intact. A typical title folder contains eboot.bin and sce_sys. Do not copy only eboot.bin.

OPTIONAL

1.6 Optional: Orbit Store

Orbit Store can be installed as a convenience layer for downloads and sources. Treat it as optional: the core setup does not depend on it. Use only content and sources you are legally entitled to access.

02

REMOTE PLAY

2. Mirror and Control the PS5 from a PC

This step is worth doing early. When the PS5 screen is visible in chiaki-ng on your PC, you can take clean screenshots, copy error messages, and show an AI exactly what the console is doing.

1

Enable Remote Play on the PS5.

2

In chiaki-ng, add or select the discovered console.

3

Choose PS5, provide the PSN Account-ID (Base64) and the Remote Play PIN, then register.

4

If PSN-based registration is not available on your jailbroken setup, use the offline workflow below.

Why do this early?

Clean screenshots, readable errors, and much faster troubleshooting from the PC.

Fallback

If PSN registration is unavailable in your jailbreak environment, use the local ActRemoteLink workflow on the next page.

02.2

OFFLINE PAIRING

2.2 Offline/jailbreak workflow with ActRemoteLink

ActRemoteLink: local registration when normal PSN pairing is unavailable.

Back up saves before changing account activation state

ActRemoteLink can alter a local account_id. The project itself warns that changing account identity can break the association with existing saves. The safest approach is to leave an already activated primary account alone and use a secondary local user for offline activation if needed.

Required files

Required release files: actremotelink_agent.elf, actremotelink_pin_notify.elf, actremotelink_sender.py.

PS5 requirement

The PS5 needs an ELF loader reachable on port 9021.

Run from the ActRemoteLink folder

```
python actremotelink_sender.py --host <PS5_IP> start

python actremotelink_sender.py --host <PS5_IP> --no-send current

python actremotelink_sender.py --host <PS5_IP> pin
```

02.2

REGISTER CONSOLE

chiaki-ng → Register Console

Use the values returned by ActRemoteLink; do not invent or reuse stale pairing data.

Field	Value
Host	<PS5_IP>
PSN Account-ID	The Base64 value returned by ActRemoteLink
Remote Play PIN	The PIN returned by the pin command
Console PIN	Optional
Console	PS5

What current gives you

The active local user and its Base64 account ID.

What pin gives you

A real Remote Play PIN displayed on the PS5 for pairing.

02.2

TROUBLESHOOTING

Problems we hit and what they meant

The failures here are usually file-version or account-state problems.

Agent sent, but it did not respond

Old or mismatched ActRemoteLink files. Replace the agent, PIN notifier and sender together from the same current release.

Cannot fake-signin on slot 1 (protection active)

Expected safety protection. Do not force it. Use a secondary local user only if fake sign-in is actually necessary.

The terminal seems frozen after the pin command

The PIN payload can stay active while waiting for pairing. Pair from chiaki-ng, wait for timeout, or use a second terminal.

Registration works but Remote Play connection fails immediately

Check whether the target user is genuinely activated/signed-in enough for Remote Play. ActRemoteLink documents an optional fake-signin workflow for a secondary offline user.

chiaki-ng asks about PSN

If you are doing local manual registration without PSN, choose No and continue with the manual fields.

03

EMULATORS

3. Universal Emulator Installation Rules

Four rules that prevent most wasted time and unnecessary SSD copying.

Installable build vs source tree

If the extracted folder contains mainly .github, src, cmake, docs, Makefile and CMakeLists.txt, you almost certainly downloaded source code. A ready PS5 title usually contains a PPSAxxxxx folder with eboot.bin, sce_sys and runtime assets.

1

Install and test one emulator at a time. Do not change the game library, keys and BIOS at the same time as the application install.

2

Keep large libraries on USB whenever the emulator officially supports USB. Do not copy 100+ GB to the internal SSD just to test a path.

3

Use exFAT for normal PC-readable USB storage. FAT32 has a 4 GB single-file limit; NTFS is not readable by the PS5 in the normal homebrew workflow.

4

After changing HEN/daemon settings, fully close and reopen the emulator. A title already running will not automatically gain new sandbox privileges.

04

NINTENDO SWITCH

4. Nintendo Switch — ProsperoEden

Confirmed working in the tested setup.

Status: Confirmed working — tested end-to-end

ProsperoEden v1.000.070 (PPSA99008) loaded prod.keys, detected 238 NCA firmware files, found the Switch game library, and launched successfully on firmware 13.40.

Normal installation

1. Keep the complete PPSA99008 application folder on USB and register it with ShadowMountPlus.
2. Inside PPSA99008, create an assets folder with keys, firmware, roms, and updates as needed.
3. After changing files or paths, fully close ProsperoEden and reopen it so the library and system data are scanned again.

Working USB layout

1. Keys: /mnt/usb0/PPSA99008/assets/keys/prod.keys
2. Firmware: unpacked *.nca files under assets/firmware. Games stay as .xci or .nsp files under assets/roms.
3. The working compatibility path appears to the app as /app0/assets. This solved the missing-keys problem when ProsperoEden could not browse /mnt/usb0 directly.

04

PS2

4. PS2 — PS5SX2

Confirmed working in the tested setup.

Status: Confirmed working in the tested setup

PS5SX2 installed, launched, found BIOS, and was configured to use an external library rather than moving the full PS2 collection to the internal SSD.

Normal installation

1. Use the official release files PS5SX2Helper.elf and PS5SX2Installer.elf.
2. Import both payloads into Payload Manager. Keep a USB backup by using Copy to Internal.
3. Load the helper, then run the installer. The installer pulls the current application build.

BIOS & USB paths

1. Place your legally dumped PS2 BIOS in the documented BIOS location. The official default is: /data/PCSX2/bios
2. For USB libraries, current PS5SX2 builds also search these paths on a USB drive: /games/ · /PCSX2/games/ · /PS5SX2/games/
3. USB BIOS paths are similarly supported under bios/, PCSX2/bios/ or PS5SX2/bios/.

04

PS5SX2

Common issues

Most failures here are path, helper, or release-package mistakes.

The emulator opened but the shelf said “No games in /data/PCSX2/games”

Do not immediately copy the whole library to internal storage. Confirm the Helper is loaded and use one of the official USB search paths.

We nearly moved a very large PS2 library unnecessarily

Same-drive folder moves can be quick, but the better solution is to use PS5SX2’s documented USB paths. Preserve the external library.

Installer/Helper were on USB but not available in the manager

Import the ELF payloads into Payload Manager, or place them in the manager’s scanned USB payload location.

Source archive confusion

Use the release installer/helper files, not a source-code archive.

05

GAMECUBE / WII

5. GameCube / Wii — Porpoise (Dolphin)

Official USB support; sandbox visibility is the key variable.

Status: App launches. USB support is official. Sandbox access was the remaining issue in the tested 13.40 session.

Porpoise officially searches /mnt/usb0 through /mnt/usb7 and extended storage, up to four folders deep. If USB is invisible, the app is still sandboxed.

Normal installation

1. Download the latest Porpoise release ZIP, not the source archive.
2. Extract the PPSA99764 title folder and register it with your normal homebrew method. Upstream documents /data/homebrew/PPSA99764; ShadowMountPlus can also scan external homebrew paths when your setup supports that.
3. Launch Porpoise once so it creates its data folders.

Library & custom paths

1. Games can be placed in /data/porpoise/games or on USB. With automatic discovery enabled, Porpoise scans common internal paths plus /mnt/usb0–/mnt/usb7 and /mnt/ext0–/mnt/ext1.
2. For a custom location: Settings → Games → Add a game folder. Browse to the folder and add it.

05

PORPOISE

Problems we hit and solutions

Do not move a huge library until you know the problem is not sandbox/daemon access.

The folder picker only showed app0, av_contents, data, dev, download0, system_tmp and a random sandbox folder

That is the app sandbox, not the real system root. USB will not be visible yet.

Old etaHEN Toolbox would not open on firmware 13.40

Use a firmware-compatible unified etaHEN build. In the tested setup, the 11.00–13.60 unified build restored the Toolbox.

Need USB access under etaHEN

In etaHEN Toolbox → Settings, enable [Debug] Legacy Jailbreak CMD Server. Fully close Porpoise and open it again.

USB still does not appear after enabling the daemon

Do not move the library. Confirm the daemon remains enabled, etaHEN networking is active, fully terminate/relaunch Porpoise, and collect Porpoise trace/core logs. Upstream also notes that if a launch lands in the sandbox, opening Porpoise once more can help when the daemon came up late.

Want to verify whether the drive itself is mounted

Check /mnt/usb0 with Web File Manager. If Web File Manager sees the drive but Porpoise does not, the issue is sandbox/daemon access, not the disk.

Logs to collect

```
/data/homebrew/PPSA99764/trace.txt

/data/homebrew/PPSA99764/porpoise/core.log
```

06

WII U / 3DS

6. Wii U / Nintendo 3DS — PS5CEMU-HAR

JIT and app-jailbreak matter more than cosmetic settings.

Normal installation

1. Use the latest release ZIP and copy/register the PPSA99360 title folder.
2. Add PPSA99360 to etaHEN’s app jailbreak list. Upstream requires this to grant /data access and JIT memory; without JIT, Cemu can fall back to the much slower interpreter.

Games & keys

1. Wii U games: default folder /data/ps5cemu/games, or choose another readable folder in Settings → Game files.
2. Encrypted WUD/WUX dumps need your own disc keys in /data/ps5cemu/keys.txt. Decrypted content does not need those disc keys.
3. 3DS games use the Azahar side. Encrypted 3DS dumps need your own aes_keys.txt under /data/ps5cemu/azahar/sysdata.

06

PS5CEMU-HAR

Performance & key mistakes

JIT, app-jailbreak, and encrypted-file keys are the three things to verify first.

Downloaded folder was full of source files (.github, docs, patches, Makefile)

That was source code. Download the release ZIP that contains PPSA99360.

App runs but Wii U performance is abnormally poor

Verify PPSA99360 is actually app-jailbroken and receiving JIT memory. Running only with a sandbox elevator can leave Cemu on its interpreter.

Keys confusion

Only encrypted dumps need the relevant keys. Do not invent a BIOS requirement for Cemu/Azahar if upstream does not require one for your file format.

07

XBOX 360

7. Xbox 360 — PS5X360

A readable mount is required; adding a folder does not mount USB for you.

Normal installation

1. Download the current installable release, not Source code. The ready title is PPSA50011.
2. Register the complete title folder. Keep eboot.bin, sce_module, sce_sys and assets together.

Games & external storage

1. Xbox 360 does not use a traditional user-supplied console BIOS in this project. Upstream explicitly ships no games, BIOS or keys.
2. For external storage, open Settings → Game folders and add a readable path. Upstream includes defaults such as /mnt/usb0/xbox360 when visible.

AutoLog ELF mistaken for the emulator

PS5X360-AutoLog is a diagnostic payload, not the emulator title. The emulator is the PPSA50011 release folder.

No games listed

Use one complete folder per game, verify the chosen game path, then refresh the game list.

USB path saved but unavailable

The app can only scan mounts it can access. The game-folder feature does not unlock or mount USB by itself.

Source tree downloaded instead of build

Use the release ZIP; a source tree with src/cmake/docs is not the installable title.

08

ORIGINAL XBOX

8. Original Xbox — XPSemu

This one genuinely needs system files from your own Xbox.

Normal installation

1. Install/register PPSA97358.
2. Load xpsemu_tools.elf together with kstuff, and follow the project’s whitelist requirement.
3. Games must be in XISO-compatible form. A full redump disc image may need conversion before XPSemu can use it.

Required system files

1. XPSemu does require your own original Xbox system files. Upstream documents: /data/xemu/mcpx_1.0.bin · /data/xemu/Complex_4627.bin · /data/xemu/xbox_hdd.qcow2 · /data/xemu/games/
2. Games must be XISO-compatible.

08

XPSEMU

Common failures

This emulator is stricter because the required system files really matter.

App closes or says XPSemuTools failed

Confirm kstuff + xpsemu_tools.elf are loaded and PPSA97358 is in the required whitelist.

Xbox Dashboard button does nothing

A blank HDD image does not include Microsoft’s dashboard. The dashboard must already exist on your own HDD image.

BIOS confusion

Unlike Xbox 360/PS5X360, XPSemu really does require MCPX ROM, flash BIOS and HDD image from your own Xbox.

09

CLASSIC SYSTEMS

9. RetroArch / PSP and Classic Systems

RetroArch / PSP: simple internal layout, but external storage is currently limited.

Normal installation

1. Install/register the complete PPSA99169 title folder.
2. Put ROMs/content under the title's content folder and BIOS/system files under system/.

PSP & other cores

1. PPSSPP is included as a core and does not need a PSP BIOS. Put PSP .iso/.cso/.pbp files under content/.
2. Some other systems do require BIOS files. Follow each core's documented system path rather than assuming every emulator needs a BIOS.

Internal layout

```
/data/homebrew/PPSA99169/content/

/data/homebrew/PPSA99169/system/
```

External storage is currently hidden from PS5 RetroArch releases

The current public release notes explicitly state that EXTERNAL is empty because the PS5 app sandbox hides USB and extended storage. Keep RetroArch content under the title's internal content folder unless a newer release explicitly changes this.

09

PS3 STATUS

PS3 status

Treat PS3 emulation as experimental/source-build territory, not a normal install target.

PS3 status

Do not download random “RPCS3 for PS5” archives and assume they are ready releases. Current PS5 RetroArch releases intentionally exclude RPCS3; upstream notes both stability and licensing/distribution constraints. Treat PS3 emulation as experimental/source-build territory, not a normal end-user install step.

What not to do

Do not download random “RPCS3 for PS5” ZIPs and assume they are ready builds.

Current reality

Public PS5 RetroArch releases intentionally exclude RPCS3 for stability and licensing/distribution reasons.

10

STORAGE

10. Storage Strategy That Avoids Wasting the Internal SSD

Keep large libraries external only where the emulator officially supports it.

Emulator	External USB?	Recommended strategy
PS5SX2	Yes	Keep large PS2 library in one of PS5SX2's documented USB folders.
Porpoise	Yes, official	Keep GameCube/Wii library on exFAT USB; solve sandbox/daemon access instead of copying it internally.
PS5X360	Yes when mount is visible	Add a USB game folder from Settings → Game folders.
PS5CEMU-HAR	Can use any folder the PS5 can read	Prefer external only after app-jailbreak/JIT and filesystem visibility are confirmed.
XPSemu	Official guide uses /data/xemu	Follow upstream internal layout unless a newer release explicitly adds external paths.
RetroArch	Not in current public release	Keep content internal; EXTERNAL is currently hidden by sandbox.

11

CHECKLIST

11. Fast Troubleshooting Checklist

Run this list before changing three things at once.

- ✓ Did I download the release asset, not GitHub Source code?
- ✓ Is the correct etaHEN build loaded for my firmware?
- ✓ Is ShadowMountPlus running and scanning the folder where I placed the title?
- ✓ Did I fully close and reopen the emulator after changing HEN/daemon settings?
- ✓ Does the emulator actually require a BIOS, firmware or keys? Check upstream before hunting for files that do not exist.
- ✓ Does the title folder contain eboot.bin and sce_sys?
- ✓ Is kstuff running if this emulator needs it?
- ✓ Can Web File Manager see /mnt/usb0? If yes but the emulator cannot, suspect sandbox/daemon access.
- ✓ Am I solving the app first before moving a huge game library?
- ✓ If a problem persists, collect the emulator's own log before changing more settings.

Best debugging habit

Change one variable at a time, reproduce once, then collect the emulator log before making another change.

12

END STATE

12. Suggested “Ultimate Home Console” End State

A clean daily workflow is more valuable than installing everything at once.

WebKit Autoloader
daily entry

→

Payload Manager
launch / import

→

etaHEN + kstuff
services / JIT

→

Homebrew apps
emulators / tools

- ✓ WebKit Autoloader → Payload Manager as the normal entry path.
- ✓ Web File Manager available from a phone/PC so the USB drive can stay connected to the PS5.
- ✓ chiaki-ng paired to the PS5 so troubleshooting and AI-assisted support can happen from the PC.
- ✓ A clean backup folder on the PC containing current payload releases, emulator release ZIPs, saves, and a copy of this guide.
- ✓ Unified etaHEN appropriate for the installed firmware, with kstuff and only the services you need.
- ✓ ShadowMountPlus for title registration and external homebrew scanning.
- ✓ Large emulator libraries left on external storage wherever the emulator officially supports it.

REF

REFERENCES

Official Project References

Always re-check the release notes before updating. PS5 homebrew changes quickly, and a step that is correct for one firmware or release can become obsolete.

PS5 WebKit Autoloader https://github.com/itsPLK/ps5-webkit-autoloader	**PS5 Payload Manager** https://github.com/itsPLK/ps5-payload-manager	**etaHEN** https://github.com/etaHEN/etaHEN
etaHEN Unified 11.00-13.60 https://github.com/GronedWaffel/etahen-11.00-13.60	**ShadowMountPlus** https://github.com/jakovasaur/ShadowMountPlus	**ActRemoteLink** https://github.com/francoataffarel/ActRemoteLink
chiaki-ng https://streetpea.github.io/chiaki-ng/	**PS5SX2** https://github.com/Swordpdf/PS5SX2	**Porpoise** https://github.com/elripalda/Porpoise-Dolphin-Emulator-for-PS5
PS5CEMU-HAR https://github.com/premohq/PS5CEMU-HAR	**PS5X360** https://github.com/BrinooTk/PS5X360	**XPSemu** https://github.com/ZiZc3/XPSemu
PS5 RetroArch https://github.com/mihawk-99/PS5_RetroArch	**GoldHEN (PS4 only)** https://github.com/GoldHEN/GoldHEN	